Министерство науки и высшего образования Российской Федерации

Федеральное государственное автономное образовательное
учреждение высшего образования
Национальный исследовательский Нижегородский государственный
университет им. Н.И. Лобачевского

Институт информационных технологий, математики и механики

Отчет по лабораторной работе №1

«Множества на основе битовых полей»

Выполнил:
студент группы 3825Б1ФИсп1
Столбова А.Д
Проверил:
Ивлев А.Д.

Нижний Новгород
2026

Содержание

Введение 3
1. Постановка задачи 4
2. Руководство пользователя 5
3. Руководство программиста 8
3.1. Описание структуры программы 8
3.2. Описание структур данных 8
3.3. Описание алгоритмов 9
4. Результаты экспериментов и тестирования 13
Заключение 15
Литература 16
Приложение 17

Введение

Конечное множество целых чисел из диапазона от 0 до N-1 удобно хранить в виде характеристического вектора: i-й бит вектора равен единице тогда и только тогда, когда число i принадлежит множеству. Такое представление занимает всего N бит, а теоретико-множественные операции превращаются в поразрядные: объединение — в «или», пересечение — в «и», дополнение — в «не». Благодаря этому битовые поля применяют там, где нужно компактно хранить и быстро обрабатывать много небольших множеств.

Битовые поля — это структуры данных, в которых каждый элемент занимает ровно один бит, что позволяет хранить до 32 булевых значений в одном машинном слове типа unsigned int.

Множество — это абстрактная структура данных для хранения неупорядоченной коллекции уникальных элементов.

В настоящей работе структура реализована двумя классами: **TBitField** — упакованный массив битов фиксированной длины и **TSet** — множество, построенное поверх битового поля.

Правильность реализации проверяется автоматическими модульными тестами на фрэймворке Google Test. Цель данной работы — разработка структуры данных для хранения множеств с использованием битовых полей, а также освоение фрэймворка для разработки автоматических тестов Google Test.

1. Постановка задачи

Цель работы — разработать структуру данных для хранения множеств с использованием битовых полей и освоить фрэймворк Google Test для создания автоматических тестов.

Для достижения цели необходимо решить следующие задачи:

1. Реализовать класс битового поля TBitField в соответствии с интерфейсом из файла tbitfield.h;
2. Реализовать класс множества TSet в соответствии с интерфейсом из файла tset.h; множество должно храниться как характеристический вектор в объекте TBitField;
3. Написать тесты на Google Test, покрывающие основной функционал и краевые случаи, и добиться их успешного выполнения.

Класс	**Требуемые компоненты**
TBitField	Конструктор по длине, конструктор копирования, деструктор; GetLength, SetBit, ClrBit, GetBit; операторы ==, !=, =, |, &, ~; потоковые операторы >> и <<; закрытые методы GetMemIndex и GetMemMask
TSet	Конструктор по мощности универса, конструктор копирования, конструктор из TBitField, оператор преобразования к TBitField; GetMaxPower, InsElem, DelElem, IsMember; операторы ==, !=, =, + int, - int, + TSet, * TSet, ~; потоковые операторы >> и <<

Таблица 1 — Необходимый к реализации интерфейс

2. Руководство пользователя

Сборка и запуск тестов. В сборку входят файлы tbitfield.cpp, tset.cpp, Test.cpp и Lab_1.cpp. Файл Test.cpp содержит модульные тесты на Google Test, файл Lab_1.cpp — демонстрационную программу, показывающую базовые сценарии использования классов. Для сборки требуются компилятор C++ и библиотека Google Test. По окончании проведения тестов в консоль выводится список выполненных тестов и итоговая строка [PASSED] 43 tests. При ошибке выводится имя упавшего теста и сравниваемые значения (рис. 1).

```
[----------] Global test environment tear-down
[==========] 43 tests from 2 test suites ran. (59 ms total)
[  PASSED  ] 43 tests.
```

Рис. 1. Результат запуска модульных тестов.

Работа с битовым полем. Поле создаётся конструктором TBitField(len). Все биты в нём изначально равны нулю, а допустимые номера битов — от 0 до len − 1. При выводе в поток поле печатается строкой из символов 0 и 1, слева находится старший бит (номер len − 1), справа — бит с номером 0. Операции |, &, ~ возвращают новое поле, операнды не изменяются.

```cpp
#include <iostream>
#include "..\..\..\includes\Lab1\tbitfield.h"

int main() {
    TBitField f(10), g(10);
    f.SetBit(1); f.SetBit(4); f.SetBit(9);
    g.SetBit(4); g.SetBit(5);

    std::cout << "f = " << f << '\n';
    std::cout << "g = " << g << '\n';
    std::cout << "f | g = " << (f | g) << '\n';
    std::cout << "f & g = " << (f & g) << '\n';
    std::cout << "~f = " << ~f << '\n';
}
```

Рис. 2. Работа с битовым полем.

```
f = 1000010010
g = 0000110000
f | g = 1000110010
f & g = 0000010000
~f = 0111101101
```

Рис. 3.Вывод программы работы с битовым полем.

Работа с множеством. Множество создаётся конструктором TSet(mp). Его универс — целые числа от 0 до mp − 1, в начале множество пусто. Операции + и - с числом, а также +, *, ~ над множествами возвращают новое множество и не меняют исходные. Объединение и пересечение определены только для множеств с одинаковой мощностью универса. Множество печатается в виде { 1, 5, 11 }, пустое множество — в виде { }.

```cpp
#include <iostream>
#include "..\..\..\includes\Lab1\tset.h"

int main() {
    TSet a(12), b(12);
    a.InsElem(1); a.InsElem(5); a.InsElem(11);
    b.InsElem(5); b.InsElem(6);

    std::cout << "a = " << a << '\n';
    std::cout << "b = " << b << '\n';
    std::cout << "a + b = " << (a + b) << '\n';
    std::cout << "a * b = " << (a * b) << '\n';
    std::cout << "~a = " << ~a << '\n';
    std::cout << "a - 5 = " << (a - 5) << '\n';
    std::cout << "a + 0 = " << (a + 0) << '\n';
}
```

Рис. 4. Работа с множеством.

```
a = { 1, 5, 11 }
b = { 5, 6 }
a + b = { 1, 5, 6, 11 }
a * b = { 5 }
~a = { 0, 2, 3, 4, 6, 7, 8, 9, 10 }
a - 5 = { 1, 11 }
a + 0 = { 0, 1, 5, 11 }
```

Рис. 5. Вывод программы работы с множеством.

Ввод данных.

Оператор	Правила чтения	Пример
in >> TBitField	Из потока читается одно слово (до пробела), состоящее из 0 и 1; первый символ записывается в старший бит (номер len − 1), следующие — в более младшие; любой символ, кроме 1, считается нулём; символы сверх длины поля игнорируются; не заданные символами младшие биты обнуляются; перед чтением все биты поля сбрасываются	Поле длины 10, ввод 0110000001 даёт поле 0110000001; ввод 101 даёт 1010000000
in >> TSet	Читаются целые числа, разделённые пробелами; чтение прекращается на первом отрицательном числе или при ошибке чтения; перед чтением множество очищается; число не меньше мощности универса вызывает исключение	Множество с универсом 12, ввод 3 7 10 -1 даёт { 3, 7, 10 }

Таблица 2 — Форматы ввода

Обработка ошибочных ситуаций. Классы сообщают об ошибках стандартными исключениями из <stdexcept>.

Ситуация	Исключение	Где возникает
Отрицательная длина поля или отрицательная мощность множества	std::invalid_argument	TBitField(len), TSet(mp)

Ситуация	Исключение	Где возникает
Номер бита или элемент вне диапазона от 0 до длины − 1	std::out_of_range	SetBit, ClrBit, GetBit, InsElem, DelElem, IsMember, + int, - int
Объединение или пересечение множеств с разной мощностью универса	std::invalid_argument	TSet + TSet, TSet * TSet
При вводе множества прочитан элемент не меньше мощности универса	std::out_of_range	in >> TSet

Таблица 3 — Исключения

3. Руководство программиста

3.1. Описание структуры программы

Программа состоит из шести файлов. Заголовочные файлы заданы условием и не изменялись. Вся реализация вынесена в .cpp-файлы.

Файл	Назначение	Содержимое
tbitfield.h	Объявление битового поля (дано)	typedef unsigned int TELEM; класс TBitField с полями BitLen, pMem, MemLen; дружественные потоковые операторы
tbitfield.cpp	Реализация TBitField	Константа bits_in_el, определения всех методов и операторов, включая operator>> и operator<<
tset.h	Объявление множества (дано)	Класс TSet с полями MaxPower и BitField; дружественные потоковые операторы
tset.cpp	Реализация TSet	Определения конструкторов, методов, операторов; все операции делегируются полю BitField
Test.cpp	Модульные тесты	Вспомогательная функция GetRandomInt и 43 теста: 21 для TBitField, 22 для TSet
Lab_1.cpp	Демонстрационная программа	Функция main

Таблица 4 — Состав программы

Зависимости между модулями. tset.h подключает tbitfield.h, tset.cpp использует оба заголовка, Test.cpp — оба класса.

3.2. Описание структур данных

Тип TELEM и константа bits_in_el. Слово битового поля — TELEM, то есть unsigned int. Константа bits_in_el = sizeof(TELEM) * 8 задаёт число бит в слове (32 при четырёхбайтовом unsigned int).

Класс TBitField хранит последовательность битов в динамическом массиве слов.

Поле	Тип	Назначение
BitLen	int	Длина поля в битах; допустимые номера битов — от 0 до BitLen − 1
pMem	TELEM*	Указатель на массив слов, в котором упакованы биты
MemLen	int	Число слов в массиве; вычисляется по формуле MemLen = BitLen / 32 + 1 (целочисленное деление)

Таблица 5 — Поля класса TBitField

Бит с номером n хранится в слове pMem[n / 32] на позиции n % 32, причём младшие номера занимают младшие разряды слова. Формула для MemLen выделяет на одно слово больше необходимого, если длина кратна 32 (в том числе при нулевой длине): избыточное слово не читается и не влияет на результат операций.

Класс TSet представляет множество чисел из универса {0, 1, …, MaxPower − 1}.

Поле	Тип	Назначение
MaxPower	int	Мощность универса — максимальное число элементов; элементами могут быть числа от 0 до MaxPower − 1
BitField	TBitField	Характеристический вектор: бит i равен 1 тогда и только тогда, когда число i принадлежит множеству

Таблица 6 — Поля класса TSet

3.3. Описание алгоритмов

Обозначения. n — длина битового поля BitLen; m — число слов MemLen = n / 32 + 1, то есть m порядка n / 32; для двух полей длины обозначаются n1 и n2. Оценки сложности приводятся по времени в худшем случае; константное число операций, не зависящее от n, обозначается O(1).

Адресация бита. Метод GetMemIndex(n) возвращает n / 32 — номер слова, а GetMemMask(n) — маску 1 << (n % 32), у которой единица стоит на позиции бита внутри слова (единица имеет тип TELEM, поэтому сдвиг не приводит к знаковому переполнению). На них построены три операции доступа: SetBit выполняет pMem[i] |= mask, ClrBit — pMem[i] &= ~mask, GetBit — (pMem[i] & mask) != 0 с приведением результата к 1 или 0. Перед обращением все три метода проверяют условие 0 ≤ n < BitLen и при нарушении бросают std::out_of_range. В каждом случае выполняется фиксированное число арифметических операций, поэтому сложность O(1).

Создание, копирование и присваивание. Конструктор проверяет len ≥ 0 (иначе std::invalid_argument), вычисляет MemLen, выделяет массив и обнуляет его циклом по m словам. Конструктор копирования выделяет новый массив и копирует в него m слов. Оператор присваивания сначала проверяет самоприсваивание; если длины полей различаются, освобождает старый массив, перенимает BitLen и MemLen и выделяет новый, после чего копирует m слов; при одинаковой длине память переиспользуется без перераспределения. Во всех трёх случаях выполняется один проход по m словам.

Сравнение. Оператор == сначала сравнивает длины (разные длины — сразу результат 0), а затем бит за битом сравнивает значения GetBit(i) для i от 0 до n − 1, завершаясь на первом расхождении. Для равных полей выполняется n итераций по O(1). Сложность в худшем случае O(n). Оператор != возвращает отрицание ==.

Операции «или» и «и». Оператор | создаёт результат длиной max(n1, n2), заполненный нулями, и затем переносит в него все единицы обоих операндов: для каждого i от 0 до n1 − 1 и от 0 до n2 − 1 проверяется бит и при необходимости устанавливается соответствующий бит результата. Недостающие биты более короткого поля считаются нулями. Сложность складывается из создания результата O(max(n1, n2) / 32) и двух циклов по n1 и n2 итераций, то есть O(n1 + n2) = O(max(n1, n2)). Оператор & создаёт результат длиной min(n1, n2) и устанавливает бит i, только если он равен 1 в обоих операндах; цикл выполняет min(n1, n2) итераций, сложность O(min(n1, n2)).

Отрицание. Оператор ~ работает пословно: создаёт поле той же длины и записывает в каждое его слово инверсию соответствующего слова источника. После инверсии неиспользуемые старшие биты последнего слова, бывшие нулями, становятся единицами и нарушили бы инвариант класса. Поэтому вычисляется tail = n % 32, и если tail ≠ 0, последнее слово умножается поразрядно на маску (1 << tail) − 1, оставляющую младшие tail бит. Проверка tail ≠ 0 необходима, чтобы не выполнять сдвиг на ширину типа, поведение которого не определено. Цикл выполняет m итераций, сложность O(m).

Потоковый ввод-вывод поля. Оператор << выводит биты от n − 1 до 0, то есть старший бит слева, сложность O(n). Оператор >> читает слово из потока, сбрасывает все биты поля (n итераций) и записывает первые min(|s|, n) символов слова в биты, начиная со старшего; сложность O(n + |s|), где |s| — длина прочитанного слова. Форматы ввода и вывода зеркальны: строка, выведенная оператором <<, читается оператором >> в то же поле.

Класс TSet. InsElem, DelElem и IsMember напрямую вызывают SetBit, ClrBit и GetBit. Теоретико-множественные операции выражаются через операции поля: объединение — |, пересечение — &, дополнение — ~; результат оборачивается в новое множество конструктором из TBitField. Операторы + int и - int создают копию множества, добавляют или удаляют в ней элемент и возвращают копию; исходное множество не изменяется. Объединение и пересечение проверяют равенство MaxPower у операндов и при несовпадении бросают std::invalid_argument. Благодаря этому длины полей совпадают и правила max и min для разных длин на уровне множества не используются. Ввод множества очищает его, затем читает целые числа до первого отрицательного значения и вставляет их, а число не меньше MaxPower вызывает std::out_of_range. Вывод печатает в фигурных скобках элементы по возрастанию через запятую.

Метод	**Параметры и результат**	**Описание**	**Сложность**
TBitField(int len)	len — длина в битах (не меньше 0); создаёт поле	Проверяет len, вычисляет MemLen, выделяет и обнуляет массив	O(m): цикл обнуления делает m итераций
TBitField(const TBitField &bf)	bf — копируемое поле	Новый массив и копирование m слов	O(m)
~TBitField()	—	Освобождает массив delete[], обнуляет указатель	O(1)
GetLength()	Возвращает BitLen	Возвращает длину поля в битах	O(1)
SetBit(n)	n — номер бита; ничего не возвращает; out_of_range при n вне [0, BitLen)	Включает бит n	O(1)
ClrBit(n)	То же	Выключает бит n	O(1)
GetBit(n)	n — номер бита; возвращает 0 или 1; out_of_range вне границ	Читает бит n	O(1)
operator==	bf — второй операнд; возвращает 1, если длины и все биты совпадают, иначе 0	Сравнивает длины, затем биты по порядку до первого расхождения	O(n): до n итераций по O(1); O(1) при разных длинах
operator!=	То же, результат противоположный	Отрицание ==	O(n)
operator=	bf — присваиваемое поле; возвращает *this	При самоприсваивании ничего не делает; при разной длине перераспределяет память; копирует слова	O(m)
operator|	bf — второй операнд; возвращает новое поле длиной max(n1, n2)	Единицы обоих полей переносятся в результат	O(n1 + n2): два цикла по n1 и n2 итераций

Метод	Параметры и результат	Описание	Сложность
operator&	bf — второй операнд; возвращает новое поле длиной min(n1, n2)	Бит результата равен 1, если он равен 1 в обоих операндах	O(min(n1, n2))
operator~	Возвращает новое поле той же длины	Инвертирует слова и обнуляет неиспользуемые биты последнего слова	O(m): один проход по словам
operator>>	Поток и поле; возвращает поток	Читает слово из 0 и 1, первый символ — в старший бит	O(n + |s|)
operator<<	Поток и поле; возвращает поток	Печатает n символов, от старшего бита к младшему	O(n)
GetMemIndex(n), GetMemMask(n)	n — номер бита; индекс слова и маска	n / 32 и 1 << (n % 32)	O(1)

Таблица 7 — Методы и операторы класса TBitField

Метод	Параметры и результат	Описание	Сложность
TSet(int mp)	mp — мощность универса; создаёт пустое множество	MaxPower = mp, поле создаётся длиной mp	O(m)
TSet(const TSet &s)	s — копируемое множество	Копирует MaxPower и поле	O(m)
TSet(const TBitField &bf)	bf — характеристический вектор	Множество с MaxPower, равным длине bf, и копией bf	O(m)
operator TBitField()	Возвращает TBitField	Возвращает копию характеристического вектора	O(m)
GetMaxPower()	Возвращает MaxPower	Мощность универса	O(1)
InsElem(Elem), DelElem(Elem)	Elem — число из [0, MaxPower); out_of_range вне диапазона	Вызывают SetBit и ClrBit	O(1)
IsMember(Elem)	Elem — число; возвращает 1 или 0	Вызывает GetBit	O(1)
operator==, operator!=	s — второе множество; возвращают 1 или 0	Сравнивают MaxPower, затем поля	O(n), n = MaxPower
operator=	s — присваиваемое множество; возвращает *this	Проверка самоприсваивания, копирование MaxPower и поля	O(m)

Метод	Параметры и результат	Описание	Сложность
operator+(int Elem)	Elem — добавляемый элемент; возвращает новое множество	Копия *this с вставленным элементом	O(m): копирование
operator-(int Elem)	Elem — удаляемый элемент; возвращает новое множество	Копия *this без элемента	O(m)
operator+(const TSet &s)	s — второе множество; возвращает объединение; invalid_argument при разных MaxPower	BitField | s.BitField	O(n): оператор | поля
operator*(const TSet &s)	s — второе множество; возвращает пересечение; invalid_argument при разных MaxPower	BitField & s.BitField	O(n): оператор & поля
operator~()	Возвращает дополнение до универса	~BitField	O(m)
operator>>	Поток и множество; возвращает поток	Очищает множество (MaxPower итераций), читает k чисел и вставляет их	O(MaxPower + k)
operator<<	Поток и множество; возвращает поток	Печатает элементы по возрастанию в фигурных скобках	O(MaxPower)

Таблица 8 — Методы и операторы класса TSet

4. Результаты экспериментов и тестирования

Методика. Корректность проверялась модульными тестами на Google Test. Каждый тест объявляется макросом TEST(Группа, Имя) и использует проверки EXPECT_EQ, EXPECT_NE, EXPECT_TRUE, EXPECT_FALSE, а для исключительных ситуаций — ASSERT_NO_THROW, ASSERT_THROW и ASSERT_ANY_THROW. Размеры полей и множеств в ряде тестов задаются функцией GetRandomInt(min, max) на основе rand().

Объём. Набор содержит 43 теста: 21 для TBitField и 22 для TSet. Из них 17 заимствованы у одногруппников (Князев Н., Марков А.) и помечены в Test.cpp комментариями с ФИО авторов. Остальные 26 тестов написаны самостоятельно (14 для TBitField и 12 для TSet).

№	Тест	Что проверяет
1	Test_Create_Negative_Length	TBitField(-5) бросает std::invalid_argument
2	Test_Create_Zero_Length	Создание поля нулевой длины не бросает исключений
3	Test_Get_Length	GetLength возвращает длину, с которой создано поле (случайная длина 100–400)
4	Test_Operator_Equal	Два поля одной длины с одинаково установленным битом равны
5	Test_Operator_Not_Equal	Поля одной длины с разными установленными битами (5 и 10) не равны
6	Test_Operator_Assign	После присваивания поле равно источнику, биты 5 и 10 установлены
7	Test_Copy_Constructor	Копия равна исходному полю, биты 3 и 7 сохранены
8	Test_Multiple_Bits	В поле длины 64 установлены все чётные биты; проверяются все 64 бита, в том числе на границе слов (31 и 32)
9	Test_Length_One	Поле длины 1: бит 0 устанавливается и читается, обращение к биту 1 бросает исключение
10	Test_Length_Multiple_Of_32	Поле длины 32: крайние биты 0 и 31 устанавливаются и читаются
11	Test_Length_33	Поле длины 33: биты 0 и 32 лежат в разных словах и читаются корректно
12	Test_Not_With_Partial_Last_Element	Поле длины 5 с битами 0 и 2: после ~ установлены биты 1, 3, 4 и сброшены 0, 2 — проверка неполного последнего слова
13	Test_Assign_Different_Lengths	Присваивание полю длины 20 поля длины 10 меняет длину на 10 и переносит бит 5
14	Test_Copy_Different_Lengths	Копия поля длины 10 имеет ту же длину и сохраняет бит 5

Таблица 9 — Собственные тесты класса TBitField

№	Тест	Что проверяет
1	Test_Create_Negative_Power	TSet(-10) бросает std::invalid_argument
2	Test_Operator_Equal	Множества с одинаковыми элементами {5, 10} равны
3	Test_Operator_Not_Equal	Множества {5} и {10} не равны
4	Test_Operator_Assign	После присваивания множество равно источнику, элементы 5 и 10 принадлежат ему
5	Test_Copy_Constructor	Копия равна исходному множеству, элементы 3 и 7 сохранены
6	Test_Union_Different_Powers	Объединение множеств с мощностями 10 и 20 бросает исключение
7	Test_Intersection_Different_Powers	Пересечение множеств с мощностями 10 и 20 бросает исключение
8	Test_Empty_Set	Только что созданное множество не содержит ни одного элемента универса
9	Test_Full_Set	После вставки всех элементов все они принадлежат множеству, а дополнение пусто
10	Test_Operator_Minus_Element	Результат s - elem не содержит elem, исходное множество не изменяется
11	Test_Get_Max_Power	GetMaxPower возвращает заданную мощность универса
12	Test_Multiple_Operations	Объединение чётных и нечётных чисел 0–9 даёт все числа 0–9, а их пересечение пусто

Таблица 10 — Собственные тесты класса TSet

Краевые случаи. В наборе проверены нулевая, единичная, кратная 32 (32, 64) и некратная (33, 5) длины; границы слов 31/32; отрицательные длина и мощность; обращение к битам и элементам с номерами −1 и len; инверсия при неполном последнем слове; присваивание и копирование полей разной длины; объединение и пересечение множеств разной мощности; пустое и полное множество.

Результат прогона. Все 43 теста выполняются успешно (рис. 1) при каждом запуске, что подтверждает корректность реализации обоих классов. Успешное прохождение этих тестов на краевые случаи подтверждает, что оператор ~ корректно обнуляет неиспользуемые старшие биты последнего слова, а операторы | и & правильно работают с полями разной длины. Тесты на класс TSet подтверждают математическую корректность теоретико-множественных операций и то, что дополнение множества не выходит за пределы заданного универсума.

Заключение

В ходе работы получены следующие результаты:

1. Реализован класс TBitField по заданному интерфейсу;
2. Реализован класс TSet, хранящий множество в виде характеристического вектора;
3. Разработан набор из 43 модульных тестов на Google Test; все тесты выполняются успешно; освоены макросы TEST, EXPECT_*, ASSERT_* и проверка исключений;

Литература

Список источников, используемых при выполнении работы.

1. Страуструп Бьерн Язык программирования C++. Специальное издание. Пер. с англ. – М.: Издательство Бином, 2017 г. – 1136 с.
2. Сайт cppreference: https://en.cppreference.com/
3. Документация Google Test: https://google.github.io/googletest/

Приложение

Если необходимо, то небольшие участки кода с пояснениями, дополнительные изображения или таблицы.

Lab_1.cpp

```
#include <gtest/gtest.h>
#include <iostream>
#include "......\includes\Lab1\tset.h"
#include "......\includes\Lab1\tbitfield.h"

int main() {
    TBitField f(10), g(10);
    f.SetBit(1); f.SetBit(4); f.SetBit(9);
    g.SetBit(4); g.SetBit(5);
    std::cout << "f = " << f << '\n';
    std::cout << "g = " << g << '\n';
    std::cout << "f | g = " << (f | g) << '\n';
    std::cout << "f & g = " << (f & g) << '\n';
    std::cout << "~f = " << ~f << '\n';

    TSet a(12), b(12);
    a.InsElem(1); a.InsElem(5); a.InsElem(11);
    b.InsElem(5); b.InsElem(6);
    std::cout << "a = " << a << '\n';
    std::cout << "b = " << b << '\n';
    std::cout << "a + b = " << (a + b) << '\n';
    std::cout << "a * b = " << (a * b) << '\n';
    std::cout << "~a = " << ~a << '\n';
    std::cout << "a - 5 = " << (a - 5) << '\n';
    std::cout << "a + 0 = " << (a + 0) << '\n';
}
```